

Currículo interativo

Explicação da arquitetura, implementação visual, integração com banco de dados, players do TikTok e YouTube e carrossel horizontal.

PROJETO	Currículo web de Bruno Pinto Corrêa da Silva
STACK	React + TypeScript + Vite Node.js + Express + Prisma + PostgreSQL
DATA	03 de outubro de 2026

Este documento descreve como cada parte foi construída a partir do código atual do projeto.

1. Visão geral do projeto

O projeto é um currículo interativo em formato de página única (one-page). A interface apresenta a identidade profissional de Bruno, sua formação, experiência audiovisual, projetos de desenvolvimento, tecnologias e canais de conteúdo.

Objetivos principais

- Apresentar o perfil profissional de forma visual, clara e responsiva.
- Conectar projetos diretamente aos repositórios do GitHub.
- Carregar vídeos cadastrados no banco por meio da API do back-end.
- Exibir TikTok e YouTube dentro da própria página, sem exigir que o visitante saia do currículo.
- Organizar vídeos em carrosséis horizontais para preservar espaço e permitir navegação lateral.

Estrutura de pastas

Pasta/arquivo	Responsabilidade
front/src/App.tsx	Componente principal, dados de apresentação, projetos, filtros e vídeos.
front/src/styles.css	Layout, cores, tipografia, responsividade e carrossel.
front/index.html	Documento HTML inicial e metadados do site.
back/src/server.ts	Servidor Express e registro das rotas.
back/src/routes/videos.ts	CRUD da tabela de vídeos.
back/src/routes/imagens.ts	CRUD da tabela de imagens/conteúdos.
back/prisma/schema.prisma	Modelos Video e Imagem no PostgreSQL.

2. Front-end: inicialização e componente principal

O front-end utiliza React com TypeScript. O arquivo index.tsx cria a raiz React e renderiza o componente App. O componente importa o CSS global e concentra a composição da página.

Entrada da aplicação (front/src/index.tsx)

```
import ReactDOM from "react-dom/client";
import App from "./App";

const root = ReactDOM.createRoot(document.getElementById("root") as HTMLElement);
root.render(<App />);
```

Estado e carregamento dos dados

O React usa useState para guardar o filtro de tecnologias, vídeos e imagens. O useEffect executa uma única busca quando a página é montada. A URL do back-end vem da variável VITE_API_URL, com fallback para localhost.

Busca paralela da API

```
const API_URL = import.meta.env.VITE_API_URL ?? "http://localhost:3000";
```

```
const [videos, setVideos] = useState<MediaItem[]>(fallbackVideos);
const [images, setImages] = useState<MediaItem[]>(fallbackImages);

useEffect(() => {
  const loadMedia = async () => {
    const [videosResponse, imagesResponse] = await Promise.all([
      fetch(`${API_URL}/videos`),
      fetch(`${API_URL}/imagens`),
    ]);
    if (videosResponse.ok) setVideos(await videosResponse.json());
    if (imagesResponse.ok) setImages(await imagesResponse.json());
  };
  void loadMedia();
}, []);
```

Os arrays de fallback permitem que a seção continue apresentável quando o back-end não está em execução. Quando a API responde corretamente, os dados do banco substituem os exemplos.

3. Construção visual da página

Cabeçalho e navegação

O cabeçalho usa links de ancora para navegar pelas seções da mesma página: Sobre, Experiência, Projetos, Tecnologias e Canais. A marca visual combina o monograma B com o nome Bruno Corrêa.

Estrutura resumida do cabeçalho

```
<header className="topbar">
  <a className="brand" href="#inicio">
    <span className="brand-mark">B</span>
    <span>Bruno <b>Corrêa</b></span>
  </a>
  <nav className="desktop-nav">
    <a href="#sobre">Sobre</a>
    <a href="#projetos">Projetos</a>
    <a href="#canais">Canais</a>
  </nav>
</header>
```

Hero e apresentação

A seção hero é a primeira impressão do currículo. O texto apresenta nome, disponibilidade e posicionamento profissional. A ilustração lateral é construída somente com elementos HTML e CSS: círculos coloridos, cartões inclinados e o selo circular.

Sobre e experiência

A seção Sobre usa uma grade de duas colunas para separar o título do texto. Os números de destaque são apresentados como estatísticas visuais. A experiência é uma timeline: cada item tem período, título e descrição.

Projetos e links de repositório

Os projetos são armazenados em um array tipado. Cada objeto possui título, descrição, tecnologias, cor de destaque e URL. O elemento raiz de cada card é um link, portanto o card inteiro abre o GitHub em uma nova aba.

Tipo e renderização de um projeto

```
type Project = {
  title: string;
  description: string;
  tags: string[];
  accent: string;
  url: string;
};

<a className={`project-card accent-${project.accent}`}
  href={project.url} target="_blank" rel="noreferrer">
  <h3>{project.title}</h3>
  <p>{project.description}</p>
</a>
```

Filtro de tecnologias

A lista de grupos é criada com Set para remover duplicidades. Ao clicar em um botão, activeGroup muda e useMemo recalcula somente as tecnologias pertencentes ao grupo selecionado.

Filtro por categoria

```
const filteredTechnologies = useMemo(
```

```
() => technologies.filter((technology) =>
  activeGroup === "Todos" || technology.group === activeGroup
),
[activeGroup],
);
```

4. Responsividade e identidade visual

O CSS usa uma paleta escura com verde-lima como cor de acentuação. As fontes Manrope, Space Grotesk e DM Mono criam contraste entre texto de leitura, títulos e informações técnicas. O layout se adapta com media queries.

• Desktop: grade de duas colunas no hero, quatro colunas na grade de projetos e navegação completa.

• Tablet: hero em uma coluna e cards de projeto em duas colunas.

• Celular: projetos em uma coluna, navegação simplificada e cards de mídia menores.

• Carrossel: overflow-x: auto mantém os vídeos em uma linha e permite arrastar/rolar horizontalmente.

CSS do carrossel horizontal

```
.media-rail {
  display: flex;
  gap: 12px;
  overflow-x: auto;
  padding-bottom: 10px;
}

.media-card {
  flex: 0 0 245px;
}
```

5. Back-end e banco de dados

Servidor Express

O servidor configura CORS e JSON, registra as rotas e disponibiliza a API na porta 3000. O front-end consulta /videos e /imagens.

Registro das rotas (back/src/server.ts)

```
const app = express();
const port = 3000;

app.use(cors());
app.use(express.json());
app.use('/videos', videosRouter);
app.use('/imagens', imagensRouter);
```

Rotas de consulta

Cada rota usa Prisma para consultar a tabela correspondente. O método GET retorna todos os registros; o GET por ID localiza um item específico. POST valida o corpo com Zod antes de criar e DELETE remove o item.

Consulta dos vídeos

```
router.get("/", async (req: Request, res: Response) => {
  try {
    const videos = await prisma.video.findMany();
    res.json(videos);
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: "Erro ao buscar vídeos" });
  }
});
```

Modelo Prisma

Tabelas usadas pela aplicação

```
model Video {
  id      Int      @id @default(autoincrement())
  title   String
  url     String
  createdAt DateTime @default(now())
  @@map("videos")
}

model Imagem {
  id      Int      @id @default(autoincrement())
  title   String
  url     String
  createdAt DateTime @default(now())
  @@map("imagens")
}
```

A estrutura atual é suficiente para os embeds, porque a plataforma é identificada pela própria URL. Não foi necessário adicionar coluna de plataforma ou de ID.

6. Importação e execução dos vídeos

Separação por plataforma

Depois que a API retorna os registros, o front-end separa os itens pelo domínio da URL. Os vídeos TikTok são limitados a 13 itens e os conteúdos YouTube também são limitados a 13, conforme o requisito do

carrossel.

Classificação e limite dos canais

```
function getPlatform(url: string) {
  return url.toLowerCase().includes("tiktok")
    ? "TikTok"
    : "YouTube";
}

const tiktokItems = videos
  .filter((item) => getPlatform(item.url) === "TikTok")
  .slice(0, 13);

const youtubeItems = [...images, ...videos]
  .filter((item) => getPlatform(item.url) === "YouTube")
  .slice(0, 13);
```

TikTok: player oficial

Cada URL possui um segmento /video/ seguido do identificador numérico. A função `getTikTokEmbedUrl` extrai esse identificador e monta o endpoint oficial do player TikTok. O resultado é usado no `src` de um `iframe`.

Extração do ID e embed TikTok

```
function getTikTokEmbedUrl(url: string) {
  const videoId = url.match(/\/video\/(\d+)/)?.[1];
  return videoId
    ? `https://www.tiktok.com/player/v1/${videoId}?description=1&music_info=1`
    : null;
}

{tikTokUrl && (
  <iframe
    src={tikTokUrl}
    title={item.title}
    loading="lazy"
    allow="autoplay; encrypted-media"
    allowFullScreen
  />
)}
```

O parâmetro `description=1` habilita a descrição e `music_info=1` habilita informações da música. Os parâmetros extras existentes na URL original, como `is_from_webapp` e `sender_device`, não precisam ser enviados ao player.

YouTube Shorts: conversão para embed

As URLs do banco estão no formato `/shorts/ID?t=segundos`. A função usa a URL para ler o caminho e os parâmetros. O ID é extraído do caminho e é convertido para `start` no endereço `/embed`.

Conversão de Shorts para player YouTube

```
function getYouTubeEmbedUrl(url: string) {
  const parsedUrl = new URL(url);
  const videoId = parsedUrl.pathname.match(
    /\(?:shorts|embed\)\/([^\?]+)/
  )?.[1] ?? parsedUrl.searchParams.get("v");
  if (!videoId) return null;

  const start = parsedUrl.searchParams.get("t")?.replace(/\D/g, "");
  return `https://www.youtube.com/embed/${videoId}${
    start ? `?start=${start}` : ""
  }`;
}
```

7. Componente de mídia e carrossel

O componente MediaRail cria o título do canal, o link externo e a faixa rolável. Cada registro é renderizado por MediaCard, que escolhe o player conforme a plataforma.

Estrutura do rail

```
function MediaRail({ title, subtitle, items, channelUrl }: Props) {
  return (
    <div className="media-block">
      <div className="media-heading">
        <h3>{title} <small>{subtitle}</small></h3>
        <a href={channelUrl} target="_blank" rel="noreferrer">
          Ver canal ?
        </a>
      </div>
      <div className="media-rail">
        {items.map((item) => (
          <MediaCard item={item} key={` ${item.id}-${item.title}`} />
        ))}
      </div>
    </div>
  );
}
```

Fluxo completo de um vídeo

1. O administrador grava title e url no banco PostgreSQL.
2. Prisma consulta a tabela por meio da rota Express.
3. React faz fetch para /videos e /imagens.
4. A URL é classificada como TikTok ou YouTube.
5. O ID é extraído e transformado em uma URL de player.
6. MediaCard renderiza um iframe dentro do card.
7. CSS mantém cards verticais em uma faixa com rolagem lateral.

Exemplo de transformação

Origem no banco	Resultado no iframe
tiktok.com/@brfox.games/video/7636520015499480338	tiktok.com/player/v1/7636520015499480338
youtube.com/shorts/bM_ov7qLVxM?t=9	youtube.com/embed/bM_ov7qLVxM?start=9

Por que usar iframe?

O iframe mantém o vídeo hospedado e executado pela própria plataforma. Isso evita baixar ou armazenar os arquivos de vídeo no projeto, respeita a origem do conteúdo e permite que os controles oficiais sejam utilizados.

8. Como executar o projeto

Front-end

Comandos do front-end

```
cd C:\Users\bruno\Senac\curriculo\front
npm install
```

```
npm run dev
```

Back-end

Comandos do back-end

```
cd C:\Users\bruno\Senac\curriculo\back
npm install
npm run dev
```

Variável de ambiente

front/.env

```
VITE_API_URL=http://localhost:3000
```

O back-end deve estar conectado ao PostgreSQL por meio de DATABASE_URL. Com os dois processos ativos, o front-end deixa de usar os fallbacks e exibe os registros reais do banco.

Validação realizada

- TypeScript: aprovado com `npm exec tsc -- --noEmit`.
- Build: aprovado com `npm exec vite build`.
- Estrutura do player: TikTok e YouTube tratados por funções separadas.
- Responsividade: carrossel preservado em telas menores.

9. Observações e manutenção

O projeto foi implementado sem alterar o formato atual das tabelas. Para adicionar novos vídeos, basta inserir registros com título e URL válida. O sistema reconhece URLs TikTok com `/video/ID` e URLs YouTube em `/shorts/ID`, `/embed/ID` ou `/watch?v=ID`.

Se uma plataforma alterar as regras do player incorporado, pode ser necessário atualizar as funções de conversão. Também é possível acrescentar campos como plataforma, thumbnail e ordem de exibição em uma evolução futura, caso o conteúdo precise de gerenciamento mais avançado.

O documento descreve o código existente no momento da geração e serve como guia técnico e material de apresentação do projeto.